AKADEMIA NAUK STOSOWANYCH
W NOWYM SACZU

Wydział Nauk Inżynieryjnych

DOKUMENTACJA PROJEKTOWA

Systemy sztucznej inteligencji: analiza, projektowanie i integracja API

Oczko - Inteligentna Szafka na Listy i Dokumenty

Autorzy:
Wiktor Matusik
Damian Basta
Mateusz Kowalczyk
Serafym Bokii

Prowadzacy:
dr inż. Przemysław Pardel

Nowy Sacz 2026

Contents

1 Ogólne określenie wymagań **2**
1.1 Nazwa i cel projektu . . . 2
1.2 Użytkownicy docelowi i rozwiazywany problem . . . 2
1.3 Zasadność zastosowania Sztucznej Inteligencji (AI) . . . 2
1.4 Porównanie procesu: Tradycyjny vs. Projektowany System . . . 2

2 Analiza problemu i Architektura **3**
2.1 Wymagania Funkcjonalne i Niefunkcjonalne . . . 3
2.1.1 Wymagania Funkcjonalne (FR) . . . 3
2.1.2 Wymagania Niefunkcjonalne (NFR) . . . 3
2.2 Dane wejściowe i wyjściowe . . . 3

3 Projektowanie i Implementacja **4**
3.1 Wykorzystywane narzedzia i technologie . . . 4
3.2 Struktura repozytorium i pliku konfiguracyjnego . . . 4

4 Wnioski **4**

1 Ogólne określenie wymagań

1.1 Nazwa i cel projektu

Projekt nosi nazwe **„Oczko – Inteligentna Szafka na Listy i Dokumenty"**. Głównym celem przedsiewziecia jest stworzenie autonomicznego systemu wbudowanego przeznaczonego do automatycznego wyzwalania rejestracji, cyfrowego odczytu (OCR), klasyfikacji dokumentów papierowych oraz automatycznego generowania powiadomień e-mail przy zachowaniu pełnej kontroli czynnikiem ludzkim (*Human-in-the-loop*).

1.2 Użytkownicy docelowi i rozwiazywany problem

- **Użytkownicy docelowi:** Pracownicy sekretariatów, dziekanatów, kancelarii oraz biur obsługi klienta, którzy codziennie mierza sie z duża ilościa przychodzacej korespondencji papierowej.
- **Rozwiazywany problem:** W tradycyjnym modelu segregacja listów jest procesem czasochłonnym i podatnym na błedy ludzkie. Wymaga manualnego czytania danych odbiorcy, szukania właściwego adresu e-mail w bazie, recznego formułowania wiadomości i fizycznego umieszczania pisma w odpowiedniej przegródce.

1.3 Zasadność zastosowania Sztucznej Inteligencji (AI)

Zastosowanie nowoczesnych multimodalnych modeli LLM/VLM (*Vision-Language Model*), takich jak **Google Gemini API**, znajduje pełne uzasadnienie z nastepujacych powodów:

1. **Różnorodność formatów i układów dokumentów:** Tradycyjne algorytmy OCR wymagaja sztywnych szablonów stron. Model AI potrafi analizować niestrukturyzowane pisma o zmiennym kroju czcionki, różnym układzie nagłówków czy obecności pieczatek.
2. **Semantyczna analiza treści:** Model sztucznej inteligencji nie tylko odczytuje znaki, ale rozumie kontekst pisma – potrafi samodzielnie wywnioskować, kto jest rzeczywistym adresatem oraz jaki jest typ dokumentu (np. podanie, faktura, pismo urzedowe).
3. **Generowanie treści wiadomości:** AI automatycznie tworzy zwiezłe i poprawne jezykowo streszczenie oraz projekt wiadomości e-mail do odbiorcy.

1.4 Porównanie procesu: Tradycyjny vs. Projektowany System

Table 1: Porównanie tradycyjnego i nowego procesu dystrybucji korespondencji

Proces tradycyjny (Manualny)	**Proces w systemie Oczko**
Reczne otwieranie i czytanie każdego pisma.	Włożenie pisma do strefy skanowania i wciśniecie przycisku.
Manualne szukanie odbiorcy i właściwej przegródki.	Automatyczna sygnalizacja LED skrytki docelowej w 1–3s.
Reczne przepisywanie danych i pisanie e-maili od zera.	Automatyczne wygenerowanie gotowego szkicu e-mail.
Wysokie ryzyko pomyłek przy wysyłce powiadomień.	Jednoklikowe zatwierdzenie e-maila przez operatora.

2 Analiza problemu i Architektura

2.1 Wymagania Funkcjonalne i Niefunkcjonalne

2.1.1 Wymagania Funkcjonalne (FR)

- **FR-01 (Wyzwalanie GPIO):** Obsługa fizycznego przycisku (TACT Switch) podłaczonego do magistrali GPIO Banana Pi w celu wyzwolenia skanowania.
- **FR-02 (Przetwarzanie obrazu):** Pobranie obrazu z kamery USB i przeskalowanie go na urzadzeniu brzegowym do maksymalnej rozdzielczości 1600 px z optymalna kompresja JPEG.
- **FR-03 (Analiza VLM/OCR):** Przesłanie skanu do Google Gemini API i odebranie ustrukturyzowanej odpowiedzi w formacie JSON zawierajacej: adresata, typ pisma oraz podsumowanie.
- **FR-04 (Sygnalizacja sprzetowa):** Aktywacja odpowiedniej diody LED przypisanej do rozpoznanej skrytki odbiorcy.
- **FR-05 (Generator E-mail):** Automatyczne utworzenie gotowego szkicu wiadomości (`.eml` / `.txt`) i zapis w katalogu `./Results`.
- **FR-06 (Obsługa błedów API):** Mechanizm ponawiania prób (*retry*) i obsługi błedów sieciowych lub przeciażenia API (bład 503).

2.1.2 Wymagania Niefunkcjonalne (NFR)

- **NFR-01 (Human-in-the-loop):** Bezpieczeństwo procesu – system nie wysyła e-maili automatycznie, co eliminuje ryzyko błednej dystrybucji.
- **NFR-02 (Niska latencja):** Czas wykonania pełnej akcji (od przycisku do zaświecenia diody) nie przekracza 1–3 sekund.
- **NFR-03 (Lekkość systemu):** Zużycie pamieci RAM na jednostce Banana Pi M2 Zero utrzymuje sie poniżej 100 MB.
- **NFR-04 (Niezawodność sprzetowa):** Wyraźna sygnalizacja statusu LED oraz odporność na błedy odczytu.

2.2 Dane wejściowe i wyjściowe

- **Dane Wejściowe:**
  - Sygnał cyfrowy z fizycznego przycisku GPIO.
  - Obraz statyczny z kamery USB (surowy kadr w rozdzielczości Full HD / HD).
  - Klucz API zapisany w pliku `KEY.json`.
- **Dane Wyjściowe i Korzyści:**
  - Sygnał sterujacy GPIO zapalajacy konkretna diode LED na szafce.
  - Plik szkicu wiadomości pocztowej (`.eml`) gotowy do zatwierdzenia przez operatora.
  - Zintegrowane pliki historii w folderze `./Results`.
  - **Korzyści:** Skrócenie czasu obsługi pisma o 80%, eliminacja błedów ludzkich oraz pełna archiwizacja cyfrowa.

3 Projektowanie i Implementacja

3.1 Wykorzystywane narzedzia i technologie

- **Banana Pi M2 Zero (512 MB RAM):** Jednostka centralna uruchamiajaca system Linux.
- **Python 3.10+:** Główny jezyk programowania logiki aplikacji.
- **OpenCV & NumPy:** Biblioteki do wstepnego przetwarzania obrazu i kompresji na urzadzeniu brzegowym.
- **Google Gemini API (`google-genai`):** Chmurowy multimodalny model AI (`gemini-3.1-flash-lite` / `gemini-3.8-flash`).
- **RPi.GPIO / WiringPi:** Biblioteki sterowania fizycznymi wejściami/wyjściami mikroprocesora.

3.2 Struktura repozytorium i pliku konfiguracyjnego

Plik konfiguracyjny `KEY.json` przechowuje klucz dostepowy oraz dane adresowe

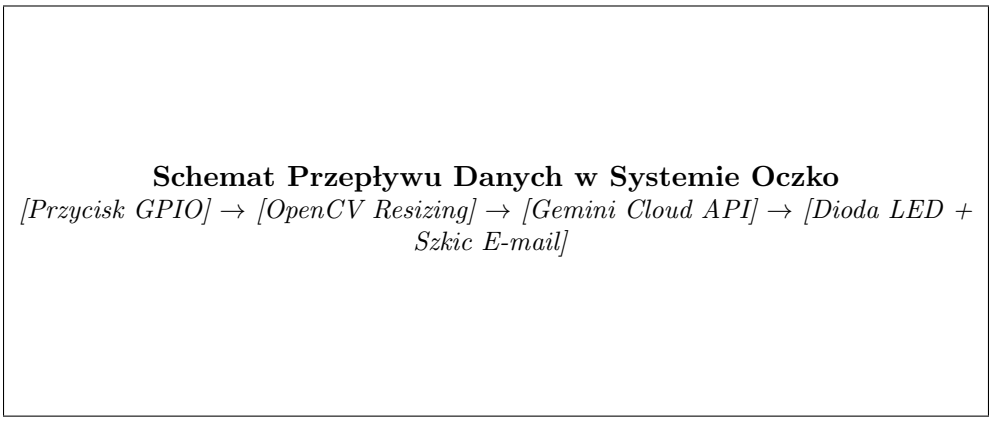


Figure 1: Architektura przetwarzania Edge Processing + AI Cloud Service.

4 Wnioski

Projekt **Oczko** udowadnia skuteczność połaczenia zoptymalizowanych urzadzeń brzegowych o niskim poborze mocy z zaawansowanymi chmurowymi modelami AI. Wprowadzenie zasady *Human-in-the-loop* pozwala wyeliminować problem błednej klasyfikacji przy jednoczesnym znacznym przyspieszeniu obróbki dokumentów.